Эмбеддинги и поиск

Как сравнивать тексты в общем пространстве

От выбора отдела к поиску ответа

Слайд 1. Эмбеддинги и поиск

Классификатор может определить, что вопрос относится к библиотеке. Но студенту нужна конкретная инструкция: например, как продлить книгу и на какой срок.

Слайд 2. Пользователю нужен документ

Теперь результатом системы станет список документов, которые стоит прочитать для ответа.

В нашей базе есть короткие тексты о библиотеке, общежитии, учебном процессе и других вопросах. Для каждого запроса нужно сравнить его с документами и расположить подходящие выше остальных. Это задача поиска и ранжирования.

Можно начать с совпадений слов. В запросе «Как продлить книгу?» и документе о продлении встречаются общие слова и их фрагменты. TF-IDF уже даёт для этого числовое представление. Но запрос может содержать «выданное издание», а документ — «библиотечная книга». Тогда буквальное совпадение слабее, хотя тема близка.

Сегодня разберём, какие представления помогают сравнивать тексты и что означает их геометрическая близость. Начнём с простых векторов, затем перейдём к плотным представлениям и соберём поиск по знакомой коллекции. Проверять будем не только число сходства, но и сам найденный документ.

Текст получает место в пространстве

Слайд 3. Смысл распределён по вектору

После TF-IDF каждый документ представлен вектором одной и той же размерности. Его координаты связаны со словарными признаками. Если слова в документе нет, соответствующий вес равен нулю. У короткого сообщения ненулевых координат обычно намного меньше, чем размер всего словаря.

Плотное представление устроено иначе. Координат может быть несколько сотен, и каждая из них обычно участвует в описании текста. При этом отдельная координата не обязана иметь очевидное название вроде «библиотека» или «недовольство». Полезная информация распределена по вектору.

Такое векторное представление объекта часто называют эмбеддингом. Объектом может быть слово, токен в контексте, предложение или документ. Когда говорят «мы получили эмбеддинг», нужно уточнить, что именно представлено и каким способом.

Слайд 4. Общее пространство векторов

Для поиска требуется общее пространство. Если документы закодированы одним преобразованием, а запросы — другим несогласованным, одинаковое число координат их не спасёт. Первая координата одного вектора может иметь совсем другой смысл, чем первая координата другого.

Представьте два географических плана с разным началом координат. Сравнивать пары чисел, не согласовав системы, неудобно. С векторами текста проблема похожа: нам нужно применять совместимые преобразования и сохранять связь каждой строки матрицы с исходным документом.

Косинус сравнивает направления

Слайд 5. Косинус сравнивает направления

Один из распространённых способов сравнить векторы — косинусная близость:

```
cos(u, v) = (u · v) / (||u|| · ||v||)
```

Числитель — скалярное произведение, знаменатель — произведение длин. Для ненулевых векторов значение находится между −1 и 1. Нас интересует направление: насколько согласованы координаты двух объектов после выбранного представления.

Посчитаем на двумерном примере. Пусть u=(1;0), v=(1;1), w=(0;1).

Слайд 6. Один косинус вручную

Скалярное произведение u и v равно 1, длина u равна 1, длина v — $\sqrt{2}$. Поэтому $\cos(u,v)=1/\sqrt{2}\approx 0{,}7071$. У u и w скалярное произведение нулевое, значит, $\cos(u,w)=0$.

Если заранее привести каждый вектор к длине 1, знаменатель больше не нужно вычислять при каждом сравнении: косинус равен скалярному произведению нормированных векторов. Это удобно, когда один запрос сравнивается сразу со всей матрицей документов.

Слайд 7. Масштаб не меняет косинус

Теперь заменим v на (10;10). Длина увеличилась в десять раз, скалярное произведение с u — тоже. Их отношение осталось прежним. Поэтому

положительное масштабирование вектора не меняет косинус. Вектор стал длиннее, но сохранил направление.

Слайд 8. Нулевой запрос требует решения

Нулевой вектор — отдельный случай: его длина равна нулю, и формула не определена. Для TF-IDF такой запрос может получиться, если ни один его фрагмент не встретился в словаре. Вместо случайного ранжирования равных нулевых оценок наша функция вернёт пустую выдачу.

Геометрическая близость сама по себе ещё не является вероятностью правильного ответа. Её смысл зависит от представления и обучающей задачи. Вектор документа на похожую тему может оказаться близким к вектору запроса, даже если в документе отсутствует нужное число или условие. В конце занятия мы увидим такой случай на вопросе о стипендии.

Как SVD делает представление компактнее

Слайд 9. SVD выбирает направления

Попробуем перейти от разреженных векторов с большим числом словарных признаков к плотным векторам с небольшим числом координат. Truncated SVD выбирает несколько направлений, которые описывают существенную часть структуры исходной матрицы. Документы проецируются на эти направления.

Запишем преобразование явно. SVD представляет матрицу как $X=U\Sigma V^{T}$. Для сокращения оставляем k направлений, соответствующих наибольшим сингулярным значениям. Столбцы V_k — эти направления в исходном пространстве признаков. Новые координаты документов получаются умножением $Z=XV_k$, а запрос преобразуется как $z_q=qV_k$. Направления выбираются так, чтобы приближение исходной матрицы матрицей ранга k давало наименьшую сумму квадратов ошибок восстановления.

Слайд 10. Проекция сохраняет выбранную ось

Посмотрим на маленькую матрицу из трёх строк: X=((2;0), (0;1), (2;0)). Первая координата даёт сумму квадратов $2^2+0^2+2^2=8$, вторая — $0^2+1^2+0^2=1$; их скалярное произведение равно нулю. При сокращении до одного направления SVD оставляет первую ось, $V_1=(1;0)$, с точностью до знака. Проекция даёт три числа: 2, 0 и 2. Например, $(0;1)\cdot(1;0)=0$. Восстановить вторую координату из этих чисел уже нельзя: второй документ стал нулевым. Здесь знаки и направления особенно простые; в общем случае направление сочетает несколько исходных признаков.

Этот пример объясняет и обработку нового запроса. Для q=(1;1) получим $qV_1=1$. Нельзя обучить для него новое направление и сравнить результат со старыми

числами документов: они уже имеют конкретный смысл — проекцию на выбранную первую ось. После проекции ненулевые строки можно нормировать, а нулевые требуют отдельного решения.

Слайд 11. Компактность меняет информацию

Если исходная матрица X имеет размер «документы × признаки», после сокращения до k направлений получим «документы × k». В лабораторной матрица TF-IDF имеет размер 12×670: двенадцать документов и 670 признаков. После SVD с шестью направлениями получаем матрицу 12×6. Запрос должен пройти через тот же обученный преобразователь SVD, иначе его координаты не будут согласованы с документами.

Почему такое сокращение иногда помогает? Некоторые словарные признаки встречаются вместе. Например, разные формулировки библиотечной темы могут связываться через набор сопутствующих слов. Проекция объединяет статистическую структуру нескольких исходных координат.

При этом часть информации теряется. Редкий номер ошибки или точное имя может быть важным для поиска, даже если не определяет одно из крупных направлений матрицы. Поэтому компактность представления и качество выдачи — разные вопросы. Мы сравним результаты, а не объявим меньшую размерность автоматически лучшей.

После SVD длины векторов различаются, поэтому снова применим L2-нормировку перед сравнением скалярным произведением. Если проекция запроса оказалась нулевой, его также нужно обработать отдельно.

В этом способе плотные координаты получены линейным преобразованием нашей матрицы. Это хороший учебный мост к эмбеддингам, но не то же самое, что большой нейросетевой энкодер, обученный на других корпусах. Позже мы сравним оба подхода по их фактическому поведению.

От окружения слова к Word2Vec

Слайд 12. Окно создаёт обучающие пары

Другая идея — учить представления по окружениям слов. Слова, употребляющиеся в похожих контекстах, получают связанные обучающие сигналы. Так можно получить полезные векторы без разметки каждого слова по тематической категории.

Возьмём предложение «Студент читает новую книгу». Выберем слово «читает» и соседние слова. В варианте skip-gram обучающая задача состоит в том, чтобы по центральному слову предсказывать элементы его окружения. Из большого числа

таких пар модель подбирает представления, позволяющие различать характерные и менее характерные сочетания.

Пусть окно включает по одному слову слева и справа. Для центра «читает» получаются две положительные пары: («читает», «студент») и («читает», «новую»). «Книгу» в это окно не входит: между ним и центром два шага. Если увеличить радиус до двух, появится и эта пара. Поэтому размер окна определяет, какие соседства превращаются в обучающие ответы.

Слайд 13. Skip-gram и CBOW меняют задачу

В Word2Vec есть также вариант CBOW: по окружению предсказывается центральное слово. Эти задачи устроены иначе, чем классификация обращений. Обучающий ответ создаётся из самого текста, из положения слов в нём.

В том же примере CBOW получает окружение «студент», «новую» и должен предсказать «читает». Сравним направления предсказания: у skip-gram стрелки идут от центра к соседям, у CBOW — от представления соседей к центру. Это разные направления обучающей задачи на одном фрагменте корпуса.

Слайд 14. Учимся различать пары

Для большого словаря сравнивать каждую пару со всеми возможными словами дорого. В обучении с negative sampling настоящая пара противопоставляется выбранным отрицательным примерам. Мы не будем реализовывать этот алгоритм в обязательной работе, но важно понимать, откуда берётся сигнал для векторов: он связан с наблюдаемыми окружениями.

Например, положительной паре («читает», «студент») можно противопоставить выбранную для этого шага пару («читает», «замок»). В обучении используются векторы центральных и контекстных слов: их скалярное произведение превращается сигмоидой в оценку принадлежности пары к наблюдаемым соседствам. Потеря поощряет большую оценку для положительной пары и меньшую для выбранной отрицательной. Это не утверждение, что сочетание с «замком» невозможно в языке: здесь оно выступает отрицательным примером конкретного шага.

Полученный статический словарь обычно хранит один вектор на слово. У «ключа» этот вектор общий для разных употреблений. Можно увидеть смешение нескольких характерных окружений, но конкретное значение в данном предложении ещё не выбрано.

Близкие векторы отражают близость употребления, которая не всегда совпадает с синонимией. «Горячий» и «холодный» встречаются в похожих грамматических конструкциях и рядом с одними и теми же существительными. Это полезно помнить, когда список ближайших соседей содержит слово с противоположным значением.

Подслова помогают работать со словоформами

Слайд 15. Подслова помогают новой форме

Статическому словарю трудно обработать слово, которого в нём нет. Можно заменить его общим UNK-вектором, но тогда разные неизвестные слова становятся неразличимыми. Для русского языка это особенно заметно из-за множества словоформ.

fastText использует представления символьных фрагментов слова. Общие части позволяют связать формы вроде «книга» и «книгой». Для нового слова можно собрать представление из известных подсловных элементов, если они были изучены при обучении.

Это не морфологический разбор в школьном смысле. Границы символьной n-граммы не обязаны совпадать с границами приставки, корня или окончания. Модель использует повторяющиеся фрагменты как обучаемые признаки.

У этого подхода есть связь с символьными признаками предыдущей лабораторной. Там мы непосредственно подавали веса фрагментов классификатору. Здесь части слова участвуют в построении плотного вектора. Общая идея — использовать регулярности внутри написания, но полученные представления и обучающие задачи различаются.

Подсловная структура помогает не во всех случаях. Похожие по написанию слова могут иметь разные значения, а непохожие — описывать одну тему. Поэтому качество поиска всё равно нужно проверять на запросах и найденных документах.

Представление слова может зависеть от предложения

Слайд 16. Контекст меняет вектор позиции

Контекстная модель получает не одно слово, а последовательность. Представление «ключа» в «ключе от комнаты» может отличаться от его представления в «ключе доступа». Окружение участвует непосредственно в вычислении вектора для данного употребления.

Слайд 17. От текста к вектору документа

Это меняет и единицу, которую мы сравниваем. Статический вектор относится к слову в словаре. Контекстный — к позиции в конкретном входе. А для поиска по документам нужен ещё способ получить один вектор из последовательности таких представлений.

Можно усреднить векторы доступных токенов или использовать специальную позицию, если модель была подготовлена к такому использованию. Но способ агрегации должен соответствовать выбранной модели. На шестом занятии мы посмотрим на конкретный энкодер, его токенизатор и рекомендованный способ получить представление текста.

Различайте три вопроса: что подано модели, векторы каких объектов она вернула и какой вектор мы используем в поиске. Без этого легко назвать одним словом «эмбеддинг» несколько разных массивов и сравнить несопоставимые вещи.

В сегодняшней лабораторной плотное представление будет проще: TF-IDF и SVD. Благодаря этому весь индекс строится на двенадцати документах, а влияние сокращения размерности можно проследить без загрузки нейросети.

Индекс и запрос должны совпадать по координатам

Слайд 18. Запрос проходит тот же путь

Соберём поиск по шагам. Сначала берём известные документы, соединяем заголовок и текст, обучаем TF-IDF и сохраняем полученную матрицу. Для второго варианта обучаем SVD на этой матрице и нормируем строки.

Каждая строка должна оставаться связанной со своим `doc_id`. Если перемешать документы отдельно от векторов, можно получить правильное число сходства рядом с чужим заголовком. Поэтому индекс хранит не только числа, но и соответствие между представлением и источником.

Новый запрос проходит через тот же TF-IDF. Для плотного варианта после него применяется тот же SVD и та же нормировка. Здесь выполняется `transform`, а не новое обучение. Затем считаем близость со всеми строками индекса и сортируем оценки по убыванию.

Если оценки равны, нужен понятный порядок, например по `doc_id`. При сортировке используем исходные числа, а округление оставляем для показа. Иначе разные близкие оценки после округления могут стать равными и случайно поменять порядок результатов.

Почему сейчас можно обучить представление по всем двенадцати документам? Потому что эти документы — заранее известная база, по которой системе разрешено искать. Проверочные вопросы и их эталонные ответы в индекс не включаются. Мы проверяем, как система отвечает на новые запросы к доступной коллекции.

Это отличается от подготовки классификатора предыдущего занятия, где test содержал новые объекты, предназначенные для оценки переноса. Разделение данных определяется постановкой. Важно понимать, какая информация доступна системе до появления проверочного запроса.

Качество выдачи зависит от ранга

Слайд 19. Ранг меняет вклад в метрику

Предположим, у каждого ответимого вопроса есть один размеченный релевантный документ. Recall@k показывает долю вопросов, для которых этот документ оказался среди первых k результатов. Для k=1 проверяется только верхняя позиция, для k=3 — первые три.

MRR@k учитывает также ранг. Если эталон на первом месте, вклад равен 1; на втором — 1/2; на третьем — 1/3. Если его нет в первых k, вклад равен нулю. Затем вклады усредняются по вопросам.

Посчитаем пример из четырёх ответимых вопросов. Ранги эталонов: 1, 3, вне top-3 и 2.

Слайд 20. Попадание и ранг

Тогда Recall@1=1/4=0,25; Recall@3=3/4=0,75. MRR@3 равна `(1 + 1/3 + 0 + 1/2)/4 ≈ 0,4583`.

Два метода могут иметь одинаковую Recall@3, но разную MRR@3: один чаще ставит правильный документ первым, другой — третьим. Для пользователя это заметное различие, особенно если он читает только верхний результат.

Вопросы, для которых в базе вообще нет ответа, нужно рассматривать отдельно. У них нет эталона, который можно найти в top-3. Если молча включить их в тот же подсчёт, мы смешаем две задачи: поиск известного источника и определение отсутствия сведений.

В нашей таблице тридцать вопросов, из них двадцать четыре с ответом. Для валидации используются восемь ответимых вопросов, для теста — шестнадцать; остальные нужны для разбора отсутствия ответа. В отчёте важно указывать эти знаменатели, чтобы было понятно, по скольким случаям вычислена метрика.

Что рассказывают конкретные запросы

Слайд 21. Место при нулевой оценке

Запрос Q04 «Какие сведения нужны при технической ошибке?» нашёл документ D02 третьим в TF-IDF и первым после SVD в валидационном разборе. Откройте выдачу и сам документ: нужные сведения там действительно перечислены. Так можно понять, что означает изменение ранга для ответа пользователю.

У этого примера есть важная деталь. Первые три результата TF-IDF: D05 с оценкой около 0,0855, D01 с 0,0807 и D02 с нулём. После SVD оценка D02 составляет около

0,7899. В выдаче TF-IDF оценка D02 равна нулю. На третье место он попал среди равных нулевых кандидатов по правилу сортировки `doc_id`, после двух документов с положительной оценкой. Поэтому попадание в top-3 здесь не доказывает полезного лексического совпадения с D02. После SVD он получил положительную оценку близости и поднялся на первое место. Ранг, исходная оценка и текст документа вместе рассказывают больше, чем одно попадание.

Слайд 22. Разница — одно попадание

В лабораторной мы сравним TF-IDF и TF-IDF→SVD(6). На тестовых ответимых вопросах у обоих методов Recall@1 в исходном опыте равна 0,875. При расширении выдачи до трёх документов у TF-IDF Recall@3 около 0,9375, у SVD — 1. Эти числа полезно сопоставить с теми запросами, где порядок изменился.

Слайд 23. Близость не добавляет фактов

Другой вопрос касается суммы стипендии. Точная формулировка Q26 — «Сколько рублей составляет академическая стипендия?». Нужного числа в нашей базе нет, однако SVD находит документ D11 с косинусом около 0,9909. Высокое сходство не добавляет в документ отсутствующую сумму. Этот случай особенно важен перед RAG: генератору потребуется различать полезный источник и документ, который просто относится к похожей теме.

Иногда векторы показывают на двумерной картинке. Это удобная иллюстрация: можно отметить вопросы, документы и группы. Но исходные сравнения выполняются в полном пространстве. При проекции на две координаты часть различий теряется, поэтому близость точек на рисунке нельзя использовать вместо метрики выдачи.

Смысл работы — связать числа с найденным содержимым. Выберите запрос, проследите его преобразование, посмотрите на верхние документы и объясните, где есть ответ. Затем возьмите вопрос без ответа и покажите, почему самый высокий балл не решает проблему.

На следующем занятии мы перейдём от сравнения готовых текстов к предсказанию продолжения. Вероятностная языковая модель будет назначать оценки следующему токену, а мы разберём, как из них получается последовательность.

Для дальнейшего чтения

– Mikolov et al. Efficient Estimation of Word Representations in Vector Space.

– Mikolov et al. Distributed Representations of Words and Phrases.

– Bojanowski et al. Enriching Word Vectors with Subword Information.

– scikit-learn: TruncatedSVD.